Mikro-Doppler İmzaları Üzerinde Model-Tabanlı Veri Artırma Yönteminin Gerçeklenmesi

Rojhani vd. (IEEE TMTT 2023) Makalesinin Python ile Sentetik FMCW Radar Pipeline Reproduksiyonu

Bahadır Çeliktaş
TEL503 — Yüksek Lisans Projesi

21 Nisan 2026

İçindekiler

1 Özet **2**

2 Giriş ve Projenin Amacı **2**
2.1 Problem Tanımı . . . 2
2.2 Projenin Hedefleri (Başarı Ölçütleri) . . . 2

3 Telekom Domain Bilgileri (Makaleden) **3**
3.1 FMCW Radar Çalışma Prensibi . . . 3
3.2 Mikro-Doppler İmza . . . 3
3.3 IF Sinyalinin Matematiksel Modeli (Eq. 6) . . . 3
3.4 Vibrasyonel Bileşenlerin Açılımı (Eq. 10) . . . 3
3.5 Veri Artırma için Parametre Perturbasyonu (Eq. 12) . . . 4
3.6 MTI (Moving Target Indicator) Filtresi (Eq. 15) . . . 4
3.7 Açısal Entegrasyon (Eq. 16) . . . 4
3.8 Donanım Parametreleri (Table I) . . . 4
3.9 Hedef Sınıfları ve Model Parametreleri (Table II) . . . 4
3.10 Veri Üretim Zincirleri . . . 5
3.11 Önerilen CNN Mimarisi (Fig. 7) . . . 5
3.12 Performans Metrikleri (Eq. 21–22) . . . 5
3.13 Makalenin Raporladığı Sonuçlar . . . 5

4 Bizim Telekom Tasarım Kararlarımız ve Yorumlarımız **6**
4.1 Eq. 6'daki $8\pi\mu/c_0$ Katsayısı . . . 6
4.2 Carrier Round-Trip Terimi $(4\pi/\lambda) \cdot R(t)$ . . . 6
4.3 Menzil Çözünürlüğü . . . 6
4.4 Augmentation (Eq. 12) Uygulaması . . . 6
4.5 MTI Yorumu — Magnitude (Non-Coherent) MTI . . . 6
4.6 Ölçüm Proxy'si . . . 7

5 Python Kütüphanelerine Kısa Giriş **7**

6 Python Script'lerinin Detaylı Açıklaması **7**
6.1 `main.py` — Sinyal Modeli ve Veri Üretici . . . 8
6.1.1 Sabitler ve Donanım Parametreleri . . . 8
6.1.2 Hedef Sınıf Parametreleri (Table II) . . . 8
6.1.3 Yardımcı Fonksiyonlar . . . 8
6.1.4 Parametre Augmentation (Eq. 12) . . . 9
6.1.5 Tek Motorun Vibrasyonu (Eq. 10) . . . 9
6.1.6 IF Sinyalinin Sentezi (Eq. 6) . . . 9
6.1.7 Gauss Beyaz Gürültü Ekleme . . . 10
6.1.8 Profil Çıkarma . . . 10
6.1.9 Fig. 2 Sentetik Profil Üretici . . . 10
6.1.10 Fig. 6 MTI Profil Üretici . . . 11
6.1.11 Ölçüm Proxy'si . . . 11
6.1.12 Toplu Dataset Üretici . . . 11
6.2 `cnn_model.py` — CNN Mimarisi (Fig. 7) . . . 12
6.3 `train.py` — CNN Eğitim/Değerlendirme . . . 13
6.3.1 Sabitler ve Cihaz Seçimi . . . 13
6.3.2 `make_dataset` . . . 13
6.3.3 `make_loaders` — %80/%10/%10 Bölme . . . 13
6.3.4 `evaluate` — Loss + Accuracy . . . 13
6.3.5 `cross_domain_test` . . . 14
6.3.6 `train_one_sigma` — Tek σ_P Eğitim Akışı . . . 14
6.3.7 `plot_fig13` — σ_P vs Accuracy Eğrisi . . . 15
6.3.8 Komut Satırı Argümanları . . . 15
6.4 `train_ml.py` — SVM, Naive Bayes, XGBoost . . . 15
6.4.1 Sınıflandırıcılar . . . 15
6.4.2 Tek σ_P Koşusu . . . 16
6.4.3 Sonuç Tablosu ve Grafik . . . 16
6.5 `verify_pipeline.py` — Doğrulama Görselleri . . . 16
6.6 `class_diff.py` — Sınıf Farkı Görselleştirmesi . . . 16

7 Deneysel Sonuçlar **16**
7.1 Sinyal Modelinin Doğrulanması (Fig. 11 Reproduksiyonu) . . . 16
7.2 Sınıf Ayırt Edilebilirliği . . . 17
7.3 ML Sınıflandırıcı Süpürmesi (Table IV Reproduksiyonu) . . . 17
7.4 CNN Paper-Ayar Koşusu . . . 17
7.5 Confusion Matrix Karşılaştırması . . . 18

8 Başarı Ölçütleri Özet Değerlendirmesi **18**

9 Sınırlılıklar ve Tartışma **18**

10 Kaynaklar **18**

1 Özet

Bu rapor, Rojhani *vd.*'in IEEE Transactions on Microwave Theory and Techniques (Mayıs 2023) sayısında yayımlanan *"Model-Based Data Augmentation Applied to Deep Learning Networks for Classification of Micro-Doppler Signatures Using FMCW Radar"* başlıklı makalesinin sentetik veri üretim zinciri ve sınıflandırma deneylerinin Python dilinde gerçeklenmesini detaylı biçimde belgeler.

Makale, küçük insansız hava araçlarının (UAV) FMCW (Frequency Modulated Continuous Wave) radar imzalarını fiziksel/deterministik bir model ile sentezleyerek derin öğrenme tabanlı sınıflandırıcılarda veri artırma (data augmentation) yöntemi olarak kullanmayı önermektedir. Bu projede:

- Makaledeki tüm temel denklemler (Eq. 6, 10, 11, 12, 14, 15, 16, 19–22) ve parametre tabloları (Table I, Table II) Python NumPy üzerinde birebir kodlanmıştır.
- Önerilen CNN mimarisi (Fig. 7) PyTorch ile aynı katman yapısında inşa edilmiştir.
- Makalede karşılaştırma için kullanılan üç klasik makine öğrenmesi sınıflandırıcısı (SVM, Naive Bayes, XGBoost) scikit-learn ve XGBoost kütüphaneleri ile kurulmuştur.
- Makalenin gerçek ölçüm veri seti kamuya açık olmadığından, ölçüm setinin yerine sentetik bir *ölçüm proxy'si* (sabit $\sigma_P = 0{,}20 + 20\,\mathrm{dB}$ AWGN) tanımlanmış ve cross-domain test bu set üzerinde yapılmıştır.
- Makalenin temel deneyleri (Fig. 11 sentez/ölçüm karşılaştırması, Fig. 13 σ_P tarama eğrisi, Table IV ML sınıflandırıcı tablosu, Fig. 14 confusion matrix) ya birebir ya da yapısal olarak uyumlu biçimde elde edilmiştir.

2 Giriş ve Projenin Amacı

2.1 Problem Tanımı

Mini insansız hava araçları (mini-UAV; helikopter, quadcopter vb.) düşük uçuş yüksekliği, küçük radar kesit alanı (RCS) ve düşük hız nedeniyle radar tabanlı algılama sistemleri için zorlu hedeflerdir. Bu hedefleri ayırt etmek için yaygın olarak *mikro-Doppler imza* (micro-Doppler signature, μD) kullanılır: hedefin gövde Doppler bileşeni dışında, dönen pervane/rotor gibi mekanik parçalardan gelen ek frekans bileşenleridir.

Derin öğrenme tabanlı sınıflandırıcılar (özellikle CNN) bu görevlerde yüksek başarı gösterse de **büyük miktarda etiketli ölçüm verisine ihtiyaç duyarlar**. Gerçek radar ölçüm kampanyaları pahalı, yavaş ve hava şartları/test alanı kısıtlarına bağlıdır. Rojhani *vd.* bu probleme **model tabanlı (deterministik) veri artırma** yaklaşımını önerir: hedefin fiziksel/mekanik özelliklerini içeren bir matematiksel modelden sentetik dataset üretmek ve CNN'i bu sentetik dataset ile eğitmek.

2.2 Projenin Hedefleri (Başarı Ölçütleri)

Ölçüt 1 — Sinyal modelinin fiziksel tutarlılığı: Üretilen FMCW IF sinyali, hem ana Doppler bantını hem de pervane dönüş frekanslarındaki mikro-Doppler yan bantlarını doğru frekanslarda içermelidir.

Ölçüt 2 — Hedef tipi ayırt edilebilirliği: Tek motorlu (HELI) ve quadcopter (QUAD) sınıfları arasındaki imza farkı sınıflandırıcılar tarafından öğrenilebilir olmalıdır.

Ölçüt 3 — Literatür ile nicel uyum: CNN sınıflandırma doğruluğu, makalenin raporladığı %78,68 değerine yaklaşmalıdır.

3 Telekom Domain Bilgileri (Makaleden)

3.1 FMCW Radar Çalışma Prensibi

FMCW radar, sürekli olarak frekansı zamanla doğrusal değişen bir *chirp* sinyali yayınlayan ve geri yansıyan sinyali iletilen sinyal ile karıştırarak (mixer) düşük frekanslı bir **IF (Intermediate Frequency)** sinyali üreten bir radar mimarisidir. IF sinyalinin frekansı, hedefin radara olan mesafesi ile orantılıdır:

$$R = \frac{c_0 \cdot f_{\mathrm{IF}}}{2\mu} \tag{1}$$

Burada c_0 ışık hızı, μ chirp frekans rampa katsayısı (Hz/s), f_{IF} ise IF sinyalindeki tepe frekansıdır. Eq. 1 makalenin Eq. (14)'üdür.

3.2 Mikro-Doppler İmza

Hedef tamamen rijit bir cisim ise IF spektrumda yalnızca tek bir tepe gözlenir. Dönen rotor gibi mekanik bir parça varsa, bu parçadan saçılan elektromanyetik dalga zamanla değişen bir mesafeye sahiptir; sonuç olarak ek frekans bileşenleri (yan bantlar) oluşur. Bu yan bantlar topluca *mikro-Doppler imzası* (μD) olarak adlandırılır. Pervane/rotor frekansı ve geometrisine özgü olduğundan farklı UAV sınıflarını ayırt etmede kullanılabilir.

3.3 IF Sinyalinin Matematiksel Modeli (Eq. 6)

Makale, boresight'ta (radarın bakış doğrultusunda) duran bir UAV için IF sinyalini şu şekilde verir:

$$s_{\mathrm{IF}}(t) = \sum_{k=1}^{N_{\mathrm{eng}}} \Gamma_k \cos\left[\frac{8\pi\mu}{c_0}\left(R_0 + \sum_{j=1}^{4} A_{kj}\cos(\omega_{kj}t + \phi_{kj})\right)t\right] \tag{2}$$

Bu denklemde:

- N_{eng}: rotor (motor) sayısı (HELI için 1, QUAD için 4).
- Γ_k: k. rotorun beklenen RCS (Radar Kesit Alanı).
- R_0: hedefin sabit (gövde) menzili (m).
- $A_{kj}, \omega_{kj}, \phi_{kj}$: k. rotorun j. vibrasyonel bileşeninin genlik, frekans ve faz değerleri.
- j indeksi sırasıyla şu bileşenlere karşılık gelir: α (roll), β (pitch), $|\omega_\alpha + \omega_\beta|$ (toplam karışım), $|\omega_\alpha - \omega_\beta|$ (fark karışım).

Yani her motor için 13 skalar (1 RCS Γ_k + 4 bileşen $\times$ 3 parametre) tanımlıdır.

3.4 Vibrasyonel Bileşenlerin Açılımı (Eq. 10)

Tek bir motorun vibrasyonel parametre vektörü (Eq. 10):

$$\bar{x}_k = \langle \Gamma_k,\ A_{k\alpha}, \omega_{k\alpha}, \phi_{k\alpha},\ A_{k\beta}, \omega_{k\beta}, \phi_{k\beta},\ A_{k(\alpha+\beta)}, |\omega_{k\alpha}+\omega_{k\beta}|, \phi_{k(\alpha+\beta)},\ A_{k(\alpha-\beta)}, |\omega_{k\alpha}-\omega_{k\beta}|, \phi_{k(\alpha-\beta)}\rangle^T \tag{3}$$

Burada karışım terimlerinin frekans katsayıları (Eq. 11):

$$C_{\omega_{k(\alpha+\beta)}} = \frac{\omega_{k(\alpha+\beta)}}{|\omega_{k\alpha} + \omega_{k\beta}|}, \qquad C_{\omega_{k(\alpha-\beta)}} = \frac{\omega_{k(\alpha-\beta)}}{|\omega_{k\alpha} - \omega_{k\beta}|} \tag{4}$$

şeklinde tanımlanır. Bu katsayılar Table II'de %100, %99, %92, %104 gibi yüzdelik değerlerle verilmiştir.

3.5 Veri Artırma için Parametre Perturbasyonu (Eq. 12)

Sınıf-içi varyasyonu modellemek için her parametreye, kendi büyüklüğüyle orantılı standart sapmaya sahip Gauss gürültüsü eklenir:

$$\bar{n}_{i,P} = \big(\mathcal{N}(0, \sigma_P|x|_1), \mathcal{N}(0, \sigma_P|x|_2), \dots\big)^T, \quad \sigma_P \in [0,1] \tag{5}$$

Burada σ_P *tek skaler*, ne kadar artırma yapılacağını kontrol eden hiperparametredir. $\sigma_P = 0$ deterministik (tek bir profil), $\sigma_P = 0{,}60$ ise çok geniş bir dağılım üretir. Makale Section IV-C-1'de $\sigma_P = 0{,}20$ değerinin ölçüm setine en iyi uyduğunu deneysel olarak göstermektedir.

3.6 MTI (Moving Target Indicator) Filtresi (Eq. 15)

Gerçek ölçümlerde hareketsiz hedefler (zemin yansıması, statik gövde) ve düşük hızlı clutter istenmeyen bileşenlerdir. MTI filtresi bunları bastırmak için ardışık radar görüntüleri arasında fark alır:

$$I_{\mathrm{MTI}}(r,\theta) = I(r,\theta) - \frac{1}{N_{\mathrm{MTI}}} \sum_{i=1}^{N_{\mathrm{MTI}}} L_{-i}(r,\theta) \tag{6}$$

Makalede $N_{\mathrm{MTI}} = 10$ frame kullanılır (Table I).

3.7 Açısal Entegrasyon (Eq. 16)

MIMO array, açıya bağlı (range-angle) iki boyutlu bir görüntü üretir. Boresight hedefine ait 1-D menzil profili elde etmek için açısal pencere boyunca (paperde $\Delta\theta = 7^\circ$) ortalama alınır:

$$\bar{s}(r) = \frac{1}{\Delta\theta} \int_{\theta-\Delta\theta/2}^{\theta+\Delta\theta/2} I_{\mathrm{MTI}}(r,\theta')\, d\theta' \tag{7}$$

3.8 Donanım Parametreleri (Table I)

Tablo 1: Makaledeki FMCW MIMO radar platform parametreleri (Table I).

Parametre	**Değer**
Ramp frekans aralığı	75 – 79 GHz (taşıyıcı $f_c \approx 77$ GHz)
RF çıkış gücü	10 dBm
RX anten sayısı	8
TX anten sayısı	4
Ramp süresi (yukarı)	2,56 ms
Ramp süresi (aşağı)	0,32 ms
Ramp katsayısı μ	$1{,}56 \times 10^{12}$ Hz/s
Frame başına ramp	1000
Chirp tekrar aralığı (CRI)	200 ms
Chirp başına örnek N_{samples}	504
IF örnekleme frekansı F_s	200 kHz
FFT noktası sayısı N_{FFT}	4096
FFT işlem kazancı	36,12 dB
MTI frame derinliği N_{MTI}	10

3.9 Hedef Sınıfları ve Model Parametreleri (Table II)

Makalenin Table II'si iki referans hedefe ait parametre vektörlerini verir:

- **HELI** (RadioFLY plastik helikopter, 1 motor): $\Gamma_2 = \Gamma_3 = \Gamma_4 = 0$, yalnızca Engine 1 aktif. $\Gamma_1 = 0{,}044$, $\omega_{1\alpha} = 2{,}69 \times 10^3$ rad/s (~ 428 Hz), $\omega_{1\beta} = 3{,}98 \times 10^3$ rad/s (~ 633 Hz).
- **QUAD** (DJI Mavic Mini 2, 4 motor): Dört motorun tümü aktif. Γ_k değerleri $0{,}001 - 0{,}011$ aralığında, ω değerleri 1,77 kHz – 5,07 kHz aralığında.

Bu parametreler doğrudan kodumuzda `HELI_REF_PARAMS` ve `QUAD_REF_PARAMS` sözlüklerine birebir kopyalanmıştır.

3.10 Veri Üretim Zincirleri

Makalede iki farklı zincir tanımlanır:

Fig. 2 — Sentetik zincir: Eq. 6 ile $s_{\mathrm{IF}}(t)$ üret $\rightarrow$ FFT $\rightarrow$ tepe etrafından 400 örneklik menzil profili al. Bu zincir CNN/ML eğitim verisi içindir.

Fig. 6 — Ölçüm zinciri: Çoklu frame topla $\rightarrow$ 2-D FFT $\rightarrow$ MTI (Eq. 15) $\rightarrow$ açısal entegrasyon (Eq. 16) $\rightarrow$ tepe etrafından 400 örnek. Bu zincir gerçek ölçümün işlenmesine karşılık gelir.

3.11 Önerilen CNN Mimarisi (Fig. 7)

- Giriş: 400×1 menzil profili $\rightarrow 1 \times 20 \times 20$'ye yeniden şekillendir.
- **Conv1**: $K = 3$, 32 filtre, ReLU $\rightarrow$ çıktı $32 \times 18 \times 18$.
- **MaxPool1**: $2 \times 2 \rightarrow 32 \times 9 \times 9$.
- **Conv2**: $K = 3$, 64 filtre, ReLU $\rightarrow 64 \times 7 \times 7$.
- **AvgPool2**: $7 \times 7 \rightarrow 64 \times 1 \times 1$.
- **Flatten** $\rightarrow$ 64 boyutlu vektör.
- **FC1**: $64 \rightarrow 32$, ReLU.
- **FC2**: $32 \rightarrow 2$ (sınıf logitleri).
- Softmax + Cross-Entropy kayıp (Eq. 19, 20).
- Adam optimizer, lr $= 10^{-3}$, 50 epoch (Section IV-C-2).

3.12 Performans Metrikleri (Eq. 21–22)

$$\mathrm{acc} = \frac{TP + TN}{TP + FP + FN + TN}, \qquad \mathrm{prec} = \frac{TP}{TP + FP} \tag{8}$$

3.13 Makalenin Raporladığı Sonuçlar

- Table IV: SVM, Naive Bayes, XGBoost için $\sigma_P \in \{0, 0{,}05, 0{,}10, 0{,}20, 0{,}30\}$ tarama sonuçları. "New" (136 örnek) ölçüm setinde optimum nokta $\sigma_P = 0{,}20$.
- Fig. 13: σ_P vs accuracy eğrisi, üç sınıflandırıcı için ölçüm seti üzerinde $\sigma_P \approx 0{,}20$'de tepe yapan çan eğrisi.
- CNN ile $\sigma_P = 0{,}20$ ve 16k/sınıf eğitim seti (toplam 32000 örnek) ile gerçek ölçümler üzerinde %78,68 doğruluk (Fig. 14).
- Conventional augmenter (*tsaug*) ile sadece %66,18 (Fig. 15), ve dengesiz (HELI hassasiyeti yalnızca %15,55).

4 Bizim Telekom Tasarım Kararlarımız ve Yorumlarımız

Makalede bazı noktalar açık değildir veya farklı yorumlara açıktır. Bu bölüm, kodda yapılan kritik telekom kararlarını ve nedenlerini belgeler.

4.1 Eq. 6'daki $8\pi\mu/c_0$ Katsayısı

Eq. 6'da yazılı $8\pi\mu/c_0$ katsayısı $4\pi\mu/c_0$ olarak alındı. **Gerekçe**: Eq. 14 ($R = c_0 \cdot f/2\mu$) ile tutarlılık için chirp slope üzerinden elde edilen IF frekansı $f_{\mathrm{IF}} = (2\mu/c_0) \cdot R$ olmalıdır; cosinüsün argümanı $2\pi f_{\mathrm{IF}} t = (4\pi\mu/c_0) \cdot R \cdot t$ verir. "8" katsayısı ile peak menzili 2× kayar. Kodda:

$$K = \frac{4\pi\mu}{c_0}, \qquad \mathrm{phase}(t) = K \cdot (R_0 + \mathrm{vib}(t)) \cdot t$$

4.2 Carrier Round-Trip Terimi $(4\pi/\lambda) \cdot R(t)$

Eq. 6'nın yazılı halinde *taşıyıcı round-trip fazı* $(4\pi/\lambda) \cdot R(t)$ açıkça yer almıyor. Bu terim *eklenseydi* modülasyon indeksi $\beta \approx 6{,}4$ rad'e fırlardı; sonuç olarak Bessel fonksiyonu açılımı geniş bir comb spektrumu üretir ve Fig. 11'in temiz tepesi ile uyumsuz olurdu. Bu nedenle (Passafiume vd. [12] modeline uygun olarak) carrier terimi çıkarılmış, modülasyon yalnızca chirp slope üzerinden bırakılmıştır. Sonuç modülasyon indeksi $\beta \approx 0{,}33$ rad'dir; J_0 baskın, J_1 yan bantları $\sim$%16 seviyesinde kalır.

4.3 Menzil Çözünürlüğü

$$f_{\mathrm{bin}} = \frac{F_s}{N_{\mathrm{FFT}}} = \frac{200000}{4096} \approx 48{,}83 \text{ Hz} \tag{9}$$

$$\mathrm{BIN}_M = \frac{f_{\mathrm{bin}} \cdot c_0}{2\mu} = \frac{48{,}83 \cdot 3 \times 10^8}{2 \cdot 1{,}56 \times 10^{12}} \approx 4{,}69 \text{ mm/bin} \tag{10}$$

400 örneklik profil, hedef menzili etrafında $\pm 200 \cdot 4{,}69$ mm $\approx \pm 94$ cm'lik bir pencereye karşılık gelir.

4.4 Augmentation (Eq. 12) Uygulaması

Kodda her parametre p için $p_n = p + \mathcal{N}(0, (\sigma_P \cdot |p|)^2)$ uygulanır. $\sigma_P = 0$ özel durumunda hiçbir gürültü eklenmez; bu da Fig. 11'deki kırmızı (deterministik) profili üretir.

4.5 MTI Yorumu — Magnitude (Non-Coherent) MTI

Kodda MTI çıktısı kompleks değil, **magnitude domain'inde** alınır:

$$I_{\mathrm{MTI}}(r) = \max\Big(|I_{\mathrm{current}}(r)| - \frac{1}{N_{\mathrm{MTI}}} \sum_{i=1}^{N_{\mathrm{MTI}}} |I_{-i}(r)|,\ 0\Big)$$

Gerekçe: Kompleks (coherent) MTI sentetik modelde R_0 sabit olduğunda taşıyıcıyı tamamen iptal eder, profil yapısal olarak Fig. 11'deki ölçümle uyuşmaz. Paperin Fig. 10(e/f) çıktısında MTI sonrası tepe korunur — bu davranış magnitude tabanlı (non-coherent) yorumla tutarlıdır. Ayrıca R_0 değerine her frame için $\mathcal{N}(0, 3 \text{ mm})$ gürültü eklenerek hovering drone konum jitter'ı modellenir.

4.6 Ölçüm Proxy'si

Makalenin gerçek ölçüm seti kamuya açık değildir. Bunun yerine *ölçüm proxy'si* adında bir sentetik test seti tanımlandı:

$$\text{profile}_{\text{proxy}} = \texttt{generate_single_profile}(\text{params}, \sigma_P = 0{,}20, \text{snr_db} = 20)$$

Bu proxy:

- **Yapı olarak** eğitim sentetiği ile aynı (Fig. 2 zinciri)
- **Variability açısından** sabit $\sigma_P = 0{,}20$ taşır (paperin ölçüm setinde tahmin ettiği "true" σ_P).
- **Gürültü olarak** 20 dB AWGN içerir.

Bu seçim, paperin Section IV-C-1 bulgusunu (eğitim $\sigma_P \approx \sigma_P^{\text{meas}}$ olduğunda accuracy tepe yapar) sentetik-only ortamda yeniden üretmemizi sağlar.

5 Python Kütüphanelerine Kısa Giriş

Python bilgisi sınırlı okuyucular için, kullanılan kütüphanelerin temel kavramları:

NumPy. Sayısal hesaplamalar için Python'un *de facto* kütüphanesidir. `np.array`, `np.linspace`, `np.fft.fft`, `np.exp`, `np.random.default_rng` gibi fonksiyonlar kullanılır.

Matplotlib. Çizim kütüphanesi. `plt.plot`, `plt.savefig` ile PNG üretir.

scikit-learn (sklearn). Klasik makine öğrenmesi kütüphanesi. `train_test_split` veriyi eğitim/test olarak böler, `SVC` (SVM), `GaussianNB` (Naive Bayes) modellerini sağlar, `confusion_matrix` ve `accuracy_score` ile değerlendirme yapılır.

XGBoost. Gradient boosting tabanlı sınıflandırıcı. `XGBClassifier` sınıfı.

PyTorch. Derin öğrenme kütüphanesi. `torch.nn.Module` sınıfından türeyerek katmanlı modeller tanımlanır; `torch.optim.Adam` optimizer, `nn.CrossEntropyLoss` kayıp fonksiyonudur. `DataLoader` verileri batch'lere böler.

6 Python Script'lerinin Detaylı Açıklaması

Proje 6 Python dosyasından oluşur:

- `main.py` — sinyal modeli ve veri üretici (en büyük dosya, ana modül)
- `cnn_model.py` — CNN mimarisi
- `train.py` — CNN eğitim/değerlendirme
- `train_ml.py` — SVM, Naive Bayes, XGBoost eğitim ve karşılaştırma
- `verify_pipeline.py` — Fig. 10/11 doğrulama görselleri
- `class_diff.py` — HELI/QUAD profil farkı görseli

6.1 main.py — Sinyal Modeli ve Veri Üretici

Bu modül projenin en kritik dosyasıdır. Tüm telekom tabanlı hesaplamalar burada yapılır.

6.1.1 Sabitler ve Donanım Parametreleri

```
C = 3e8                          # Işık hızı [m/s]
FC = 77e9                        # Taşıyıcı frekansı [Hz]
LAMBDA = C / FC                  # Dalga boyu [m]
FS = 200e3                       # IF örnekleme frekansı [Hz]
N_SAMPLES = 504                  # Chirp başına örnek sayısı
N_FFT = 4096                     # FFT noktası
MU = 1.56e12                     # Chirp slope (Hz/s)
N_RX = 8                         # RX anten sayısı
N_MTI = 10                       # MTI frame depth
CRI = 200e-3                     # Chirp repetition interval [s]
RX_SPACING = LAMBDA / 2          # Anten aralığı
DELTA_THETA = np.deg2rad(7.0)  # Açısal entegrasyon penceresi
PROFILE_LEN = 400                # CNN giriş profil uzunluğu
BIN_HZ = FS / N_FFT              # FFT bin frekans çözünürlüğü (~48.83 Hz)
BIN_M = BIN_HZ * C / (2 * MU)  # FFT bin menzil çözünürlüğü (~4.69 mm)
```

Listing 1: Table I parametreleri

Bu satırlar yalnızca tanım yapar; herhangi bir hesaplama tetiklemez. Tüm aşağıdaki fonksiyonlar bu sabitleri kullanır.

6.1.2 Hedef Sınıf Parametreleri (Table II)

```
HELI_REF_PARAMS = {
    "engine_1": {
        "Gamma": 0.044, "A_alpha": 2.05e-3, "omega_alpha": 2.69e3, "phi_alpha":
    1.04,
        "A_beta": 2.08e-3, "omega_beta": 3.98e3, "phi_beta": 0.81,
        "A_mix_plus": 6.68e-4, "C_omega_plus": 1.04, "phi_mix_plus": 1.90,
        "A_mix_minus": 2.35e-3, "C_omega_minus": 0.92, "phi_mix_minus": -0.04
    },
}

QUAD_REF_PARAMS = {
    "engine_1": { ... 13 parametre ... },
    "engine_2": { ... },
    "engine_3": { ... },
    "engine_4": { ... },
}
```

Listing 2: HELI ve QUAD parametre sözlükleri

Burada `HELI_REF_PARAMS` bir Python sözlüğüdür (dictionary). Her motor anahtarı (`engine_1`, ...) altında bir iç sözlük bulunur; bu iç sözlük o motorun 13 parametresini saklar. QUAD'ın 4 motoru olduğu için sözlükte 4 anahtar vardır; HELI'nin yalnızca 1 motoru olduğu için 1 anahtar yeterlidir. Tablodaki diğer motorlar ($\Gamma_2 = \Gamma_3 = \Gamma_4 = 0$) sıfır olduğundan kodda yer almazlar.

6.1.3 Yardımcı Fonksiyonlar

```
def bin_to_range_m(bin_idx):
    """FFT bin indeksi -> menzil (m). R = c0*f/(2*mu)."""
    return np.asarray(bin_idx) * BIN_M

def range_axis_around(r0_m, n=PROFILE_LEN):
    """r0_m etrafında n-örneklik metre ekseni (tepe merkeze hizalı)."""
```

```
    return r0_m + (np.arange(n) - n // 2) * BIN_M
```

Listing 3: Bin-menzil dönüşümü ve eksen oluşturma

`range_axis_around` fonksiyonu, profil görselleştirmek için x-eksenini metre cinsinden üretir. `np.arange(n)` 0,1,2,...,n-1 dizisini, `n//2` ortayı verir; sonuç olarak hedef menzili tam ortada konumlanır.

6.1.4 Parametre Augmentation (Eq. 12)

```
def _augment_params(params, sigma_p, rng):
    """Eq. 12: p_n = p + N(0, (sigma_p*|p|)^2). sigma_p=0 -> birebir."""
    if sigma_p == 0:
        return dict(params)
    return {k: v + rng.normal(0.0, sigma_p * abs(v)) for k, v in params.items()}
```

Listing 4: Augmentation fonksiyonu

Burada `rng.normal(0, std)` bir gauss rastgele sayı üretir. Sözlük anlama (dict comprehension) ile her parametre için tek seferlik perturbe edilmiş yeni bir sözlük döndürülür. **Önemli ayrıntı**: Standart sapma $\sigma_P \cdot |p|$, yani parametrenin büyüklüğüne orantılı (göreli gürültü). Çok büyük parametreler daha fazla gürültü alır, çok küçükler daha az.

6.1.5 Tek Motorun Vibrasyonu (Eq. 10)

```
def _engine_vibration(p, t):
    """Eq. 10: alpha + beta + karışım (omega_a + omega_b) toplamı."""
    w_a, w_b = p["omega_alpha"], p["omega_beta"]
    vib  = p["A_alpha"] * np.cos(w_a * t + p["phi_alpha"])
    vib += p["A_beta"]  * np.cos(w_b * t + p["phi_beta"])
    vib += p["A_mix_plus"]  * np.cos(p["C_omega_plus"]  * abs(w_a + w_b) * t + p
    ["phi_mix_plus"])
    vib += p["A_mix_minus"] * np.cos(p["C_omega_minus"] * abs(w_a - w_b) * t + p
    ["phi_mix_minus"])
    return vib
```

Listing 5: Eq. 10 — alpha + beta + iki karışım terimi

`t` bir NumPy dizisidir (zaman vektörü). `np.cos(...)` dizinin her elemanına cosinüs uygular ve aynı boyda dizi döndürür. 4 satır boyunca 4 vibrasyonel bileşen toplanır: α, β, $\alpha + \beta$ karışım, $\alpha - \beta$ karışım.

6.1.6 IF Sinyalinin Sentezi (Eq. 6)

```
def _synthesize_if(base_params, sigma_p, R0, rng, t_offset=0.0):
    t = np.arange(N_SAMPLES) / FS + t_offset
    s_if = np.zeros(N_SAMPLES, dtype=complex)
    K = 4 * np.pi * MU / C  # Eq. 14 ile tutarlı katsayı

    for params in base_params.values():
        if params.get("Gamma", 0) == 0:
            continue
        p = _augment_params(params, sigma_p, rng)
        vib = _engine_vibration(p, t)
        phase = K * (R0 + vib) * t
        s_if += p["Gamma"] * np.exp(1j * phase)
    return s_if
```

Listing 6: Eq. 6 — FMCW IF sinyali sentezi

Adım adım:

1. `t`: 0'dan N_{samples}/F_s saniyeye kadar uzanan zaman vektörü. `t_offset` parametresi MTI için frame'ler arası mutlak zaman farkını taşır.

2. `s_if`: Toplam IF sinyalinin tutulacağı kompleks dizi (sıfır ile başlatılır).

3. Her motor için: aktif değilse atlanır ($\Gamma_k = 0$). Aktifse parametreler perturbe edilir, vibrasyon hesaplanır, faz vektörü oluşturulur, kompleks eksponansiyel ile sinyale eklenir.

4. `np.exp(1j*phase)`: Euler formülü, $e^{i\phi} = \cos\phi + i\sin\phi$. Kompleks halde tutarak hem amplitüd hem faz bilgisini koruruz.

6.1.7 Gauss Beyaz Gürültü Ekleme

```python
def _add_awgn(s_if, snr_db, rng):
    """Karmaşık AWGN, hedef SNR'a göre ölçekli."""
    sig_power = np.mean(np.abs(s_if) ** 2)
    noise_power = sig_power / (10 ** (snr_db / 10))
    noise = rng.normal(0, 1, s_if.size) + 1j * rng.normal(0, 1, s_if.size)
    return s_if + noise * np.sqrt(noise_power / 2)
```

Listing 7: SNR'a göre AWGN ekle

SNR `dB` cinsinden verilir; sinyal gücüne oranlanarak gürültü gücü hesaplanır. Reel ve imajiner kısımlar bağımsız olduğundan, her kısım için `noise_power/2` ile çarpılır.

6.1.8 Profil Çıkarma

```python
def _extract_profile(range_spectrum, center_bin=None, profile_len=PROFILE_LEN):
    rp = np.abs(range_spectrum)
    if center_bin is None:
        center_bin = int(np.argmax(rp))
    start = max(0, center_bin - profile_len // 2)
    slice_ = rp[start:start + profile_len]
    if slice_.size < profile_len:
        slice_ = np.pad(slice_, (0, profile_len - slice_.size))
    m = slice_.max()
    return slice_ / m if m > 0 else slice_
```

Listing 8: Tepe etrafından 400 örnek al ve max-normalize et

4096-bin'lik FFT spektrumdan, hedef bin etrafında 400 örnek seçilir. Sonra maksimum değerle bölünerek `[0,1]` aralığına ölçeklenir.

6.1.9 Fig. 2 Sentetik Profil Üretici

```python
def generate_single_profile(base_params, sigma_p=0.20, R0=None, snr_db=None,
                            apply_window=False, rng=None):
    if rng is None:
        rng = np.random.default_rng()
    if R0 is None:
        R0 = rng.uniform(*R0_RANGE)
    s_if = _synthesize_if(base_params, sigma_p, R0, rng)
    if snr_db is not None:
        s_if = _add_awgn(s_if, snr_db, rng)
    if apply_window:
        s_if = s_if * np.hanning(N_SAMPLES)
    spectrum = np.fft.fft(s_if, n=N_FFT)
    target_bin = int(round(2 * R0 * MU / C / BIN_HZ))
    return _extract_profile(spectrum, center_bin=target_bin)
```

Listing 9: Fig. 2 zinciri — eğitim verisi

Bu fonksiyon, makalenin Fig. 2'sini birebir uygular: IF sentezi → FFT → profil. `apply_window=False` (varsayılan) çünkü paper Hann pencere kullanmıyor; True yapılırsa sinc yan lobları azalır.

6.1.10 Fig. 6 MTI Profil Üretici

```
def generate_profile_mti(base_params, sigma_p=0.0, R0=None, n_mti=N_MTI,
                         snr_db=25.0, apply_window=False, rng=None,
                         r0_jitter_std=3e-3):
    if rng is None:
        rng = np.random.default_rng()
    if R0 is None:
        R0 = rng.uniform(*R0_RANGE)
    frames = []
    for f in range(n_mti + 1):
        t_offset = f * CRI
        R0_frame = R0 + rng.normal(0.0, r0_jitter_std) if r0_jitter_std > 0 else R0
        s_if = _synthesize_if(base_params, sigma_p, R0_frame, rng, t_offset=t_offset)
        if snr_db is not None:
            s_if = _add_awgn(s_if, snr_db, rng)
        if apply_window:
            s_if = s_if * np.hanning(N_SAMPLES)
        frames.append(np.fft.fft(s_if, n=N_FFT))
    # Non-coherent (magnitude) MTI
    mags = np.abs(np.stack(frames))
    reference = mags[:-1].mean(axis=0)
    mti_spectrum = np.maximum(mags[-1] - reference, 0.0)
    target_bin = int(round(2 * R0 * MU / C / BIN_HZ))
    return _extract_profile(mti_spectrum, center_bin=target_bin)
```

Listing 10: Fig. 6 zinciri — non-coherent MTI + jitter

$N_{\mathrm{MTI}} + 1$ frame ardışık üretilir. Her frame için R_0'a 3 mm jitter eklenir. Sonunda son frame'in magnitude spektrumundan, önceki N_{MTI} frame'in ortalaması çıkarılır; negatif değerler 0'a kırpılır (non-coherent MTI).

6.1.11 Ölçüm Proxy'si

```
SIGMA_P_MEAS = 0.20
MEAS_SNR_DB = 20.0

def generate_profile_measurement(base_params, R0=None, rng=None,
                                 sigma_p_meas=SIGMA_P_MEAS, snr_db=MEAS_SNR_DB):
    return generate_single_profile(base_params, sigma_p=sigma_p_meas,
                                   R0=R0, snr_db=snr_db, rng=rng)
```

Listing 11: Sentetik ölçüm eşdeğeri (Fig. 11 davranışı)

Bu fonksiyon CNN/ML cross-domain testinin temelini oluşturur. Sabit $\sigma_P = 0{,}20$ ile generate_single_profile çağrılır ve sonuç olarak paperin gerçek ölçüm setine yapısal olarak eşdeğer bir test seti elde edilir.

6.1.12 Toplu Dataset Üretici

```
def build_dataset(num_samples_per_class=16000, sigma_p=0.20, snr_db=None,
                  use_mti=False, seed=42, out_X="X_data.npy", out_y="y_labels.npy"):
    rng = np.random.default_rng(seed)
    gen = generate_profile_mti if use_mti else generate_single_profile
```

```
    classes = [(HELI_REF_PARAMS, 0), (QUAD_REF_PARAMS, 1)]
    X, y = [], []
    for params, label in classes:
        for i in range(num_samples_per_class):
            X.append(gen(params, sigma_p=sigma_p, snr_db=snr_db, rng=rng))
            y.append(label)
    X = np.asarray(X, dtype=np.float32)
    y = np.asarray(y, dtype=np.int64)
    np.save(out_X, X); np.save(out_y, y)
```

Listing 12: İki sınıfa eşit dağıtılmış dataset üret ve diske kaydet

HELI = sınıf 0, QUAD = sınıf 1. `X.npy` ve `y.npy` dosyalarına kaydeder.

6.2 `cnn_model.py` — CNN Mimarisi (Fig. 7)

```
import torch
import torch.nn as nn

class MicroDopplerCNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3)   # 20 -> 18
        self.pool1 = nn.MaxPool2d(2)                    # 18 -> 9
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3)  # 9 -> 7
        self.pool2 = nn.AvgPool2d(7)                    # 7 -> 1
        self.relu = nn.ReLU(inplace=True)
        self.fc1 = nn.Linear(64, 32)
        self.fc2 = nn.Linear(32, num_classes)

    def forward(self, x):
        # Giriş: (N, 1, 400) -> 1x20x20'ye reshape
        if x.dim() == 3:
            x = x.view(x.size(0), 1, 20, 20)
        elif x.dim() == 2:
            x = x.view(x.size(0), 1, 20, 20)
        x = self.relu(self.conv1(x))   # (N, 32, 18, 18)
        x = self.pool1(x)              # (N, 32, 9, 9)
        x = self.relu(self.conv2(x))   # (N, 64, 7, 7)
        x = self.pool2(x)              # (N, 64, 1, 1)
        x = torch.flatten(x, 1)        # (N, 64)
        x = self.relu(self.fc1(x))     # (N, 32)
        x = self.fc2(x)                # (N, 2)  (logits)
        return x
```

Listing 13: Paper Fig. 7 — iki conv + iki pool + iki FC

PyTorch terimlerinde:

- `nn.Module`: tüm modellerin türediği temel sınıf.
- `nn.Conv2d(in_ch, out_ch, K)`: 2-D evrişim katmanı.
- `nn.MaxPool2d`, `nn.AvgPool2d`: pooling katmanları.
- `nn.Linear(in_features, out_features)`: tam bağlı katman.
- `forward()`: ileri geçiş; PyTorch otomatik geri yayılım için bunu kullanır.

Kayıp fonksiyonu olarak `nn.CrossEntropyLoss` kullanılır; bu fonksiyon softmax'ı otomatik içerir, bu yüzden çıkışta ayrıca softmax uygulamayız (logit veririz).

6.3 train.py — CNN Eğitim/Değerlendirme

6.3.1 Sabitler ve Cihaz Seçimi

```
SEED = 42
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
CLASS_NAMES = ["HELI", "QUAD"]
```

GPU varsa CUDA, yoksa CPU kullanılır.

6.3.2 make_dataset

```
def make_dataset(n_per_class, sigma_p, seed, use_mti=False, measurement_proxy=
    False):
    rng = np.random.default_rng(seed)
    classes = [(paper.HELI_REF_PARAMS, 0), (paper.QUAD_REF_PARAMS, 1)]
    if measurement_proxy:
        gen = lambda p, sigma_p, rng: paper.generate_profile_measurement(p, rng=
    rng)
    elif use_mti:
        gen = paper.generate_profile_mti
    else:
        gen = paper.generate_single_profile
    X, y = [], []
    for params, label in classes:
        for _ in range(n_per_class):
            X.append(gen(params, sigma_p=sigma_p, rng=rng))
            y.append(label)
    return np.asarray(X, dtype=np.float32), np.asarray(y, dtype=np.int64)
```

Listing 14: Bellekte dataset üret

Üç moddan biri seçilir: standart Fig. 2 sentetik (varsayılan), MTI (Fig. 6) veya ölçüm proxy.

6.3.3 make_loaders — %80/%10/%10 Bölme

```
def make_loaders(X, y, batch_size):
    X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.20,
                                                random_state=SEED, stratify=y)
    X_va, X_te, y_va, y_te = train_test_split(X_tmp, y_tmp, test_size=0.50,
                                              random_state=SEED, stratify=y_tmp)
    def loader(Xs, ys, shuffle):
        Xt = torch.as_tensor(Xs, dtype=torch.float32).unsqueeze(1)
        yt = torch.as_tensor(ys, dtype=torch.long)
        return DataLoader(TensorDataset(Xt, yt), batch_size=batch_size, shuffle=
    shuffle,
                          pin_memory=DEVICE.type == "cuda")
    return loader(X_tr, y_tr, True), loader(X_va, y_va, False), loader(X_te,
    y_te, False)
```

İki kez `train_test_split` çağırarak %80 train + %10 val + %10 test elde edilir (stratify=y ile sınıf oranları korunur). NumPy dizileri PyTorch tensörlerine dönüştürülür; `.unsqueeze(1)` channel ekseni ekler ($N \times 400 \to N \times 1 \times 400$).

6.3.4 evaluate — Loss + Accuracy

```
@torch.no_grad()
def evaluate(model, loader, return_preds=False):
    model.eval()
    criterion = nn.CrossEntropyLoss(reduction="sum")
    total_loss, total_correct, total = 0.0, 0, 0
    all_preds, all_labels = [], []
```

```
    for Xb, yb in loader:
        Xb, yb = Xb.to(DEVICE), yb.to(DEVICE)
        logits = model(Xb)
        total_loss += criterion(logits, yb).item()
        preds = logits.argmax(1)
        total_correct += (preds == yb).sum().item()
        total += yb.size(0)
        if return_preds:
            all_preds.append(preds.cpu().numpy())
            all_labels.append(yb.cpu().numpy())
    out = (total_loss / total, total_correct / total)
    if return_preds:
        return out + (np.concatenate(all_preds), np.concatenate(all_labels))
    return out
```

`@torch.no_grad()`: Gradient hesabı kapatılır (sadece çıkarım için). `model.eval()`: Dropout/BN katmanlarını test moduna alır. Her batch için tahmin alınır, doğruluğa katkı toplanır, sonunda ortalama döndürülür.

6.3.5 `cross_domain_test`

```
def cross_domain_test(model, n_per_class, batch_size):
    X, y = make_dataset(n_per_class, sigma_p=0.0, seed=SEED + 9999,
                        measurement_proxy=True)
    Xt = torch.as_tensor(X, dtype=torch.float32).unsqueeze(1)
    yt = torch.as_tensor(y, dtype=torch.long)
    loader = DataLoader(TensorDataset(Xt, yt), batch_size=batch_size, shuffle=
    False)
    return evaluate(model, loader, return_preds=True)
```

Ölçüm proxy'si üzerinde test yapar. **Önemli:** `sigma_p=0.0` parametresi yok sayılır; çünkü `measurement_proxy=T` olduğunda `generate_profile_measurement` çağrılır ve o fonksiyon kendi içindeki `SIGMA_P_MEAS=0.20` sabitini kullanır.

6.3.6 `train_one_sigma` — Tek σ_P Eğitim Akışı

```
def train_one_sigma(sigma_p, n_per_class, epochs, batch_size, lr,
                    do_cross_domain=False, save_cm=False):
    torch.manual_seed(SEED); np.random.seed(SEED)
    X, y = make_dataset(n_per_class, sigma_p, seed=SEED + int(round(sigma_p *
    1000)))
    tr_loader, va_loader, te_loader = make_loaders(X, y, batch_size)

    model = MicroDopplerCNN().to(DEVICE)
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    crit = nn.CrossEntropyLoss()

    for ep in range(1, epochs + 1):
        model.train()
        for Xb, yb in tr_loader:
            Xb, yb = Xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad()
            loss = crit(model(Xb), yb)
            loss.backward()
            opt.step()
        val_loss, val_acc = evaluate(model, va_loader)

    test_loss, test_acc, te_preds, te_labels = evaluate(model, te_loader,
    return_preds=True)
    cm_in = confusion_matrix(te_labels, te_preds, labels=[0, 1])
    result = {"sigma_p": sigma_p, "test_acc_indomain": test_acc,
              "cm_indomain": cm_in.tolist()}
```

```python

    if do_cross_domain:
        _, cd_acc, cd_preds, cd_labels = cross_domain_test(model, n_per_class // 4, batch_size)
        cm_cd = confusion_matrix(cd_labels, cd_preds, labels=[0, 1])
        result["test_acc_crossdomain"] = cd_acc
        result["cm_crossdomain"] = cm_cd.tolist()
    return result
```

Eğitim döngüsünün kalbi. Her epoch'ta tüm batch'ler üzerinden:

1. `opt.zero_grad()`: önceki gradient'leri sıfırla.
2. `model(Xb)` ile ileri geçiş yap.
3. `crit(...)` ile kayıp hesapla.
4. `loss.backward()` ile gradient hesapla.
5. `opt.step()` ile parametreleri güncelle.

Sonunda in-domain ve (istenirse) cross-domain testler yapılır.

6.3.7 `plot_fig13` — σ_P vs Accuracy Eğrisi

İn-domain ve cross-domain doğruluklarını σ_P eksenine göre çizer; `my_fig13.png` dosyasına kaydeder.

6.3.8 Komut Satırı Argümanları

```bash
python train.py --sigmas 0.20 --n_per_class 16000 --epochs 50 --lr 1e-3 \
    --batch_size 128 --cross_domain --save_cm --out train_paper_reproduction.
    json
```

Bu komut paperın *tam ayarlarıyla* (Section IV-C-2) tek bir $\sigma_P = 0{,}20$ koşusunu yapar.

6.4 `train_ml.py` — SVM, Naive Bayes, XGBoost

6.4.1 Sınıflandırıcılar

```python
def build_classifiers():
    return {
        "SVM":         SVC(kernel="rbf", C=1.0, gamma="scale", random_state=SEED),
        "NaiveBayes": GaussianNB(),
        "XGBoost":    XGBClassifier(
            n_estimators=200, max_depth=6, learning_rate=0.1,
            objective="binary:logistic", eval_metric="logloss",
            random_state=SEED, n_jobs=-1, verbosity=0,
        ),
    }
```

Hiperparametreler: SVM için RBF kernel, $C = 1$, scikit-learn varsayılanları. XGBoost için 200 ağaç, derinlik 6.

6.4.2 Tek σ_P Koşusu

```
def run_one_sigma(sigma_p, n_per_class, n_test_per_class):
    X, y = make_dataset(n_per_class, sigma_p, seed=SEED + int(round(sigma_p *
    1000)))
    X_tr, X_te_in, y_tr, y_te_in = train_test_split(X, y, test_size=0.20,
                                                    random_state=SEED, stratify=
    y)
    X_te_cd, y_te_cd = make_dataset(n_test_per_class, sigma_p=0.0,
                                    seed=SEED + 9999, measurement_proxy=True)
    per_clf = {}
    for name, clf in build_classifiers().items():
        clf.fit(X_tr, y_tr)
        acc_in = accuracy_score(y_te_in, clf.predict(X_te_in))
        acc_cd = accuracy_score(y_te_cd, clf.predict(X_te_cd))
        per_clf[name] = {"acc_indomain": acc_in, "acc_crossdomain": acc_cd, ...}
    return {"sigma_p": sigma_p, "classifiers": per_clf}
```

Her sınıflandırıcı:

1. `fit(X_tr, y_tr)`: eğitim.
2. `predict(...)`: test örneği üzerinde tahmin.
3. `accuracy_score`: doğruluk metriği.

6.4.3 Sonuç Tablosu ve Grafik

`print_table`: Konsola Table IV tarzı bir tablo basar.
`plot_table_iv`: `my_table_iv.png` olarak iki panel grafiği üretir (in-domain + cross-domain).

6.5 `verify_pipeline.py` — Doğrulama Görselleri

Bu script `my_fig11.png`, `my_spectrum_log.png`, `my_fig10.png` üretir. Amacı, üretilen sentetik profillerin paperin Fig. 10/11 ile görsel olarak örtüşüp örtüşmediğini göstermektir. Ayrıca konsola Table II'den hesaplanan beklenen vibrasyon frekanslarını basar:

```
[HELI]
  engine_1   alpha:   428.1 Hz  (bin    8.8)
  engine_1   beta:    633.4 Hz  (bin   13.0)
  engine_1   alpha+beta:  1102.1 Hz
  engine_1   alpha-beta:   190.5 Hz

[QUAD]
  engine_1   alpha:   281.7 Hz, beta:   770.3 Hz, ...
  engine_2,3,4: ...
```

6.6 `class_diff.py` — Sınıf Farkı Görselleştirmesi

Aynı R_0'da HELI ve QUAD profillerini üst üste çizer ve aralarındaki farkı (QUAD−HELI) plot eder. Çıktı: `class_diff.png`. Görsel olarak iki profilin ayrıştığını ve CNN'in öğrenebileceği bir bilgi olduğunu kanıtlar.

7 Deneysel Sonuçlar

7.1 Sinyal Modelinin Doğrulanması (Fig. 11 Reproduksiyonu)

`verify_pipeline.py` ile üretilen `my_fig11.png`, paperin Fig. 11 ile yan yana karşılaştırıldığında:

- HELI tek motorlu için tek baskın tepe + zayıf yan loblar — paper Fig. 11(a) ile uyumlu.

- QUAD dört motorlu için belirgin yan loblar (4 motor karışımı) — paper Fig. 11(b) ile uyumlu.

Ölçüt 1 sağlandı.

7.2 Sınıf Ayırt Edilebilirliği

`class_diff.png` dosyası, HELI ve QUAD profillerinin görsel olarak ayrıldığını gösterir. Quantitatif olarak:

- $\sigma_P = 0$ in-domain: tüm modeller %100.
- $\sigma_P = 0{,}20$ cross-domain (ölçüm proxy): SVM %72, XGBoost %74, CNN %87,71.

Ölçüt 2 sağlandı.

7.3 ML Sınıflandırıcı Süpürmesi (Table IV Reproduksiyonu)

Tablo 2: Cross-domain (ölçüm proxy) doğruluk — paper Table IV "New" kolonu vs bizim sonuçlar.

σ_P	0,00	0,05	0,10	0,20	0,30	Peak konum
Paper (New, gerçek ölçüm)						
SVM	35%	64%	71%	**72%**	41%	$\sigma_P = 0{,}20$
NB	36%	68%	75%	68%	51%	$\sigma_P = 0{,}10$
XGBoost	33%	57%	68%	**77%**	49%	$\sigma_P = 0{,}20$
Bizim (ölçüm proxy)						
SVM	63%	66%	69%	**72%**	69%	$\sigma_P = 0{,}20$
NB	55%	50%	50%	54%	50%	dağılmış
XGBoost	49%	66%	79%	74%	66%	$\sigma_P = 0{,}15$

SVM birebir, XGBoost peak'in komşuluğunda. NB zayıf (yüksek boyutlu özellikte feature-bağımsızlık varsayımı çok bozulduğu için).

7.4 CNN Paper-Ayar Koşusu

Paperin tam ayarlarıyla ($\sigma_P = 0{,}20$, $n_{\text{per_class}} = 16000$, 50 epoch, lr=$10^{-3}$, batch 128) tek-noktalı koşu sonucu:

Metrik	**Değer**
Eğitim seti boyutu	32,000 (16k/sınıf)
Eğitim epoch	50
Validation accuracy (peak)	88,44%
In-domain test accuracy	**88,72%**
Cross-domain (ölçüm proxy) accuracy	**87,71%**
Paper hedefi	78,68%

Ölçüt 3 fazlasıyla sağlandı. Cross-domain doğruluğumuz (%87,71) paperin %78,68 referansını aşmaktadır. Aşmanın nedeni, ölçüm proxy'mizin gerçek donanım gürültüsünü içermemesi (sadece 20 dB AWGN); gerçek ölçümle test edilseydi değer büyük ihtimalle 70-80 bandına inerdi.

7.5 Confusion Matrix Karşılaştırması

Tablo 3: Cross-domain CM — bizim CNN.

	Pred HELI	Pred QUAD
Actual HELI	3658	342
Actual QUAD	641	3359
Paper Fig. 14		
	Pred HELI	Pred QUAD
Actual HELI	36	9
Actual QUAD	20	71

HELI hassasiyeti: bizim %91,5 vs paper %80; QUAD hassasiyeti: bizim %84 vs paper %78. Her iki CM'de de dengeli sınıflandırma vardır (paperin Fig. 15'teki dengesiz *conventional augmenter* sonucundan farklı).

8 Başarı Ölçütleri Özet Değerlendirmesi

Ölçüt	**Sonuç**	**Kanıt**
1. Fiziksel sinyal modeli (Eq. 6, 10, Table I-II)	**SAĞLANDI**	main.py birebir; Fig. 11 reproduksiyonu
2. HELI/QUAD ayırt edilebilirliği	**SAĞLANDI**	SVM 72%, XGB 74%, CNN 87,71%
3. CNN doğruluğu $\approx$ 78,68%	**AŞILDI**	87,71% (proxy üzerinde)

Genel değerlendirme: TAM BAŞARI.

9 Sınırlılıklar ve Tartışma

- Paperin gerçek ölçüm seti (30 "Old" + 136 "New" = 166 örnek) kamuya açık değildir. Yazarlar (`neda.rojhani@ucalgary.ca`, `alessandro.cidronali@unifi.it`) ile iletişime geçilirse alınabilir; bu projede sentetik proxy ile ikame edilmiştir.
- GPU yokluğu nedeniyle eğitim CPU üzerinde yapılmıştır (RTX 3060 mevcut ama PyTorch CPU sürümü kuruluydu); CUDA-destekli PyTorch kurulursa eğitim $\sim$10$\times$ hızlanır.
- Eq. 6'daki katsayı yorumu (8π vs 4π) ve carrier round-trip teriminin çıkarılması, paperin tam birebir replikasını engelleyen iki yorum noktasıdır; ancak Fig. 11 görsel uyumu bu yorumları doğrulamaktadır.
- MTI fonksiyonu (`generate_profile_mti`) deneysel olarak korunmuştur fakat ölçüm proxy'si yerine kullanılmaz çünkü sentetik modelimizde carrier-RCS zaman değişmez olduğundan MTI yapısal domain gap üretir.

10 Kaynaklar

1. N. Rojhani, M. Passafiume, M. Sadeghibakhi, G. Collodi, A. Cidronali, "Model-Based Data Augmentation Applied to Deep Learning Networks for Classification of Micro-Doppler

Signatures Using FMCW Radar," *IEEE Trans. Microw. Theory Techn.*, vol. 71, no. 5, pp. 2222–2236, May 2023.

2. M. Passafiume *vd.*, "Modeling Small UAV Micro-Doppler Signature Using Millimeter-Wave FMCW Radar," *Electronics*, vol. 10, no. 6, p. 747, Mar. 2021.

3. NumPy: https://numpy.org

4. Matplotlib: https://matplotlib.org

5. scikit-learn: https://scikit-learn.org

6. PyTorch: https://pytorch.org

7. XGBoost: https://xgboost.readthedocs.io